



Tecsup INNOVACIÓN
CON SENTIDO

Diseño y Desarrollo de Software

LABORATORIO N° 03

Colleciones



DOCENTE:

**Juan Jose León
Suiyon**

CURSO:

Programación en Móviles Avanzado

Laboratorio 03

Colecciones: Arrays, Diccionarios y Sets

Swift en Playground — Programación en Móviles Avanzado (C43166)

Docente: Juan Leon — Tecsup

Campo	Detalle
Curso	Programación en Móviles Avanzado
Laboratorio	03 — Colecciones
Semana	3
Duración	2 horas
Docente	Juan Leon — Tecsup
Prerrequisitos	Lab 02 completado
Entregable	Repositorio GitHub: ramas manual y ai-assisted

Objetivos

- Crear y manipular Arrays, Diccionarios y Sets
- Combinar colecciones con condicionales y bucles
- Implementar un carrito de compras 2.0 con colecciones
- Leer datos del usuario con `readLine()`

□ Java/Kotlin: `Array = ArrayList`, `Dictionary = HashMap`, `Set = HashSet`.

Dinámica

- **Rama manual (sin IA):** Ejercicios 1-5 con ejemplos resueltos, TODO, FIX, PREDICT y ANALYZE.
- **Rama ai-assisted (con IA):** Ejercicios 6-7. Estructura CTRFE. Comentar CADA línea. PROMPTS.md.

En la rama manual NO uses IA.

Parte 1: Checklist de Inicio

Docente: Juan Leon — Tecsup

1. Verificar cuenta: Xcode → Settings → Source Control → Accounts
2. Verificar git config en Terminal
3. Abrir repositorio clonado
4. File → New → Project → Lab03-Colecciones → DESMARCAR ☐ Git → guardar en Semana03
5. File → New → Playground → Ejercicios → guardar en Lab03-Colecciones

☐ COMMIT → `\%C` → "feat: Crear proyecto Lab03 y Playground en Semana03" → Commit → Push

6. `%+2` → clic derecho main → New Branch → manual → Create

Ejercicio 1: Arrays

Docente: Juan Leon — Tecsup

Un Array es una colección ORDENADA de elementos del mismo tipo.

Ejemplo resuelto: Registro de notas interactivo

```
// Desarrollado por: Juan Leon
// Ejercicio 1: Arrays

import Foundation

// Crear un array vacío de notas
var notas: [Double] = []

// Pedir al usuario 3 notas
for i in 1...3 {
    print("Ingrese la nota \(i):")
    let entrada = readLine() ?? ""
    let nota = Double(entrada) ?? 0
    notas.append(nota)
}

// Mostrar las notas ingresadas
print("\nNotas ingresadas: \(notas)")
print("Total de notas: \(notas.count)")

// Calcular el promedio
var suma = 0.0
for nota in notas {
    suma += nota
}
let promedio = suma / Double(notas.count)

// Mostrar resultados
print("Suma: \(suma)")
print("Promedio: \(promedio)")
print("Nota más alta: \(notas.max()!)" )
print("Nota más baja: \(notas.min()!)" )
```

☐ Salida esperada

```
Ingrese la nota 1:
15.5
Ingrese la nota 2:
18.0
Ingrese la nota 3:
12.0

Notas ingresadas: [15.5, 18.0, 12.0]
Suma: 45.5
Promedio: 15.166...
Nota más alta: 18.0
```

*var notas: [Double] = [] → Array vacío que crece con append().
readLine() ?? "" → Lee entrada del usuario. Si falla usa String vacío.
Double(entrada) ?? 0 → Convierte texto a número. Si falla usa 0.
.sorted() → Retorna NUEVO array ordenado sin modificar el original.*

Ahora tú: Completa los TODO

```
// Desarrollado por: [TU NOMBRE]

import Foundation

// ===== TODO 1: Registro de 5 alumnos =====
var alumnos: [String] = []
for i in 1...5 {
    print("Nombre del alumno \(i):")
    let nombre = readLine() ?? ""
    alumnos.append(nombre)
}
print("Alumnos: \(alumnos)")

// ===== TODO 2: Buscar un alumno =====
print("Buscar alumno:")
let buscar = readLine() ?? ""
if alumnos.contains(buscar) {
    print("\(buscar) está en la lista")
} else {
    print("\(buscar) NO está en la lista")
}

// ===== TODO 3: Notas con clasificación =====
var notasClase: [Double] = []
for i in 1...5 {
    print("Nota del alumno \(i):")
    let n = Double(readLine() ?? "") ?? 0
    notasClase.append(n)
}
var aprobados = 0
var desaprobados = 0
var sumaNotas = 0.0
for nota in notasClase {
    sumaNotas += nota
    if nota >= 13 {
        aprobados += 1
    } else {
        desaprobados += 1
    }
}
}
```

FIX: Encuentra los errores

```
// ===== FIX: 3 errores =====

var frutas = ["Manzana", "Plátano", "Naranja"]

frutas.append(7) // 1
                FIX

let colores = ["Rojo", "Azul",
"Verde"] colores.append("Amarillo") // 2
                                   FIX

let numeros = [10, 20, 30, 40, 50]
print(numeros[5]) // 3
                  FIX
```

PREDICT

```
var lista = [1, 2, 3, 4, 5]
lista.remove(at: 0)

// PREDICT 1: ____
// PREDICT 2: ____

var nombres = ["Ana", "Carlos", "Beto"]
print(nombres.sorted()) // PREDICT 3: ____
print(nombres) // PREDICT 4: ____
```

COMMIT → %C → "feat: Completar ejercicio 1 - Arrays" → Commit → Push

Ejercicio 2: Diccionarios

Docente: Juan Leon — Tecsup

Un Diccionario almacena pares clave-valor. Cada valor se accede por su clave.

Ejemplo resuelto: Directorio de contactos

```
// Desarrollado por: Juan Leon

import Foundation

var contactos: [String: String] = [:]

for i in 1...3 {
    print("\nContacto \(i) - Nombre:")
    let nombre = readLine() ?? ""
    print("Teléfono:")
    let telefono = readLine() ?? ""
    contactos[nombre] = telefono
}

print("\n==== DIRECTORIO =====")
for (nombre, telefono) in contactos {
    print("\(nombre): \(telefono)")
}

print("\nBuscar contacto:")
let buscar = readLine() ?? ""
if let tel = contactos[buscar] {
    print("Teléfono de \(buscar): \(tel)")
} else {
    print("\(buscar) no encontrado")
}
```

*[String: String] → Diccionario clave String, valor String.
[:] → Diccionario vacío.
contactos[nombre] = telefono → Agrega o reemplaza.
if let tel = contactos[buscar] → Búsqueda segura con Optional.*

Ahora tú: Completa los TODO

```
// Desarrollado por: [TU NOMBRE]

import Foundation

// ===== TODO 4: Catálogo de productos =====
var productos: [String: Double] = [:]
for i in 1...4 {
    print("Producto \(i) - Nombre:")
    let nombre = readLine() ?? ""
    print("Precio:")
    let precio = Double(readLine() ?? "") ?? 0
    productos[nombre] = precio
}
```

```
// ===== TODO 5: Mostrar catálogo =====
print("===== CATÁLOGO =====")
for (nombre, precio) in productos { print("\
(nombre): S/. \$(precio)")
}

// ===== TODO 6: Valor total =====
var valorTotal = 0.0
for (_, precio) in productos
{ valorTotal += precio
}
print("Valor total: S/. \$(valorTotal)")

// ===== TODO 7: Buscar producto =====
print("Buscar producto:")
let buscarProd = readLine() ?? ""
if let precioEncontrado = productos[buscarProd] { print("\
(buscarProd) cuesta S/. \$(precioEncontrado)")
} else {
    print("Producto no encontrado")
}
```

ANALYZE

```
var edades: [String: Int] = ["Ana": 20, "Luis": 22, "Maria": 19]
var mayores: [String] = []
for (nombre, edad) in edades
{ if edad >= 21 {
    mayores.append(nombre)
}
}
print("Mayores de 21: \$(mayores)")
```

COMMIT → `feat: Completar ejercicio 2 - Diccionarios` → Commit → Push

Ejercicio 3: Sets

Docente: Juan Leon — Tecsup

Un Set es una colección de valores ÚNICOS sin orden. Ideal para operaciones de conjuntos.

Ejemplo resuelto: Alumnos en cursos

```
// Desarrollado por: Juan Leon

import Foundation

var alumnosIOS: Set<String> = []
print("===== ALUMNOS IOS =====")
for i in 1...3
{ print("Alumno \$(i):")
  let nombre = readLine() ?? ""
  alumnosIOS.insert(nombre)
}

var alumnosAndroid: Set<String> = []
print("\n===== ALUMNOS ANDROID =====")
for i in 1...3
{ print("Alumno \$(i):")
  let nombre = readLine() ?? ""
  alumnosAndroid.insert(nombre)
}

let enAmbos = alumnosIOS.intersection(alumnosAndroid)
let todos = alumnosIOS.union(alumnosAndroid)
let soloIOS = alumnosIOS.subtracting(alumnosAndroid)
```

```
print("\n==== RESULTADOS =====")
print("En ambos: \n(enAmbos)")
print("Solo iOS: \n(soloiOS)")
print("Solo Android: \n(soloAndroid)")
print("Total únicos: \n(todos.count)")
```

*.insert() → Agrega elemento. Si ya existe, no duplica.
 .intersection() → Elementos en AMBOS (∩).
 .union() → Elementos en CUALQUIERA (∪).
 .subtracting() → Elementos en primero que NO están en segundo (-).*

Ahora tú: TODO

```
// Desarrollado por: [TU NOMBRE]

import Foundation

// ===== TODO 8: Eliminar duplicados =====
var numeros: [Int] = []
for i in 1...8
{
  print("Número \(i):")
  let n = Int(readLine() ?? "") ?? 0
  numeros.append(n)
}
print("Con duplicados: \n(numeros)")
let sinDuplicados = Array(Set(numeros)).sorted()
print("Sin duplicados: \n(sinDuplicados)")

// ===== TODO 9: Comparar asistencia =====
// Pide 4 nombres lunes, 4 martes
```

PREDICT

```
let a: Set = [1, 2, 3, 4, 5]
let b: Set = [4, 5, 6, 7, 8]
print(a.intersection(b)) // PREDICT 5: ____
print(a.union(b).count) // PREDICT 6: ____
print(a.subtracting(b)) // PREDICT 7: ____

var repetidos: Set = ["A", "B", "A", "C", "B"]
```

COMMIT → \%\%C → "feat: Completar ejercicio 3 - Sets" → Commit → Push

Ejercicio 4: Combinación de Colecciones

Docente: Juan Leon — Tecsup

Ejemplo resuelto: Sistema de notas

```
// Desarrollado por: Juan Leon

import Foundation

var notasPa: [String: Double] = [:]
var notasPb: [String: Double] = [:]

print("¿Cuántos alumnos?")
let cantidad = Int(readLine() ?? "") ?? 0

for i in 1...cantidad
{
  print("\nAlumno \(i) -")
  Nombre: let nombre =
  readLine() ?? "" print("Nota
  Pa:")
```

```

        let pb = Double(readLine() ?? "") ?? 0
        notasPa[nombre] = pa
        notasPb[nombre] = pb
    }

    print("\n==== REPORTE =====")
    var totalAprobados = 0
    var sumaGeneral = 0.0
    for (alumno, pa) in notasPa {
        if let pb = notasPb[alumno] {
            let notaFinal = 0.30 * pa + 0.70 * pb
            let estado = notaFinal >= 13.0 ? "Aprobado" : "Desaprobado" print("\n\n(alumno): Pa=\(pa) Pb=\(pb) Final=\(notaFinal) → \((estado)") sumaGeneral += notaFinal
            if notaFinal >= 13.0 { totalAprobados += 1 }
        }
    }
    print("Promedio general: \((sumaGeneral / Double(notasPa.count))")

```

TODO 10: Inventario de productos

```

// Desarrollado por: [TU NOMBRE]

import Foundation

// Pide N productos con nombre, precio y stock
// Calcula valor total y muestra stock bajo
var precios: [String: Double] = [:]
var stocks: [String: Int] = [:]

print("¿Cuántos productos?")
let n = Int(readLine() ?? "") ?? 0
for i in 1...n {
    print("Producto \(i) - Nombre:")
    let nombre = readLine() ?? ""
    print("Precio:")
    let precio = Double(readLine() ?? "") ?? 0
    print("Stock:")
    let stock = Int(readLine() ?? "") ?? 0
    precios[nombre] = precio
    stocks[nombre] = stock
}

// TODO: Calcular valor total (precio * stock)

```

COMMIT → %C → "feat: Completar ejercicio 4 - combinación" → Commit → Push

Ejercicio 5: Carrito de Compras 2.0

Docente: Juan Leon — Tecsup

Antes: 25+ variables individuales. Ahora: 3 arrays y un bucle.

```
// Desarrollado por: [TU NOMBRE]

import Foundation

// ===== CARRITO DE COMPRAS 2.0 =====
var nombres: [String] = []
var precios: [Double] = []
var cantidades: [Int] = []

// TODO 11: Pedir productos
print("¿Cuántos productos va a comprar?")
let totalProductos = Int(readLine() ?? "") ?? 0
for i in 1...totalProductos {
    print("\nProducto \(i) - Nombre:")
    nombres.append(readLine() ?? "")
    print("Precio unitario:")
    precios.append(Double(readLine() ?? "") ?? 0)
    print("Cantidad:")
    cantidades.append(Int(readLine() ?? "") ?? 0)
}

// TODO 12: Calcular subtotales
var subtotales: [Double] = []
for i in 0..
```

```

print(sep)
for i in 0..<nombres.count {
    print("\(nombres[i]) x\cantidades[i]    S/. \totalCarrito)")
}
print(sep)
print("Subtotal:                S/. \totalCarrito)")
if descPct > 0 {
    print("Descuento \totalCarrito*(descPct*100)%: -S/. \totalCarrito*(descPct*100)%")
}
print("IGV (18%):                S/. \totalCarrito*(18%)")
print(sep)
print("TOTAL:                    S/. \totalCarrito*(1+18%)")
print(sep)
print("¡Gracias por su compra, \cliente!")

```

Ticket esperado

```

=====
TICKET DE COMPRA 2.0
Cliente: María García (VIP)
=====
Laptop x1    S/. 3500.0
Mouse x2     S/. 91.0
=====
Subtotal:    S/. 3591.0
Descuento (10.0%): -S/. 359.1
IGV (18%):   S/. 581.74
=====
TOTAL:       S/. 3813.64
=====

```

COMMIT → \%%C → "feat: Completar ejercicio 5 - carrito 2.0" → Commit → Push

Push rama manual: Source Control → Push

Rama manual con mínimo 6 commits. Carrito interactivo completo.

Parte 3: Rama ai-assisted — Con IA

Docente: Juan Leon — Tecsup

7. `%+2` → doble clic main → New Branch → ai-assisted → Create

Ejercicio 6: Gestión de Notas — con IA

8. Pedir N alumnos con nombre y 3 notas → diccionario [String: [Double]]
9. Promedio por alumno, clasificar con switch (Excelente/Bueno/Aprobado/Desaprobado)
10. Estadísticas: promedio general, nota más alta/baja, % aprobados
11. Ordenar por promedio

CADA línea con comentario específico. Header: `// Desarrollado por: [TU NOMBRE]`

`COMMIT → \%%C → "feat: Ejercicio 6 - gestión de notas (IA)" → Commit → Push`

Ejercicio 7: Inventario con menú — con IA

12. Pedir N productos con nombre, precio, stock
13. Menú while: 1)Ver inventario 2)Buscar 3)Stock bajo 4)Valor total 5)Salir
14. Reportes formateados

Comentar CADA línea.

`COMMIT → \%%C → "feat: Ejercicio 7 - inventario (IA)" → Commit → Push`

Push rama ai-assisted

Parte 4: PROMPTS.md

Docente: Juan Leon — Tecsup

```
// # Prompts - Lab 03
// ## Docente: Juan Leon - Tecsup
// ## Herramienta: [Claude / ChatGPT / Copilot]
// ## Ejercicio 6 - Gestión de notas
// ### Prompt (CTRFE):
// CONTEXTO: Estudiante iOS semana 3...
// TAREA: ...
// RESTRICCIONES: Solo semana 1-3. NO struct/class...
// FORMATO: Código con comentario en CADA línea
```

`COMMIT → \%%C → "docs: Agregar PROMPTS.md" → Commit → Push`

GitHub :

https://github.com/Rony10-az/MOVILES_AVANZADO_QUINTANA.git

Ejercicio 6 :

```
===== ALUMNOS ORDENADOS POR PROMEDIO =====
Rony Jhostin: 17.333333333333332
Haro Gabriel: 13.333333333333334
```

```
// Desarrollado por: Rony Jhostin Quintana

import Foundation

// ===== EJERCICIO 6: GESTIÓN DE NOTAS =====

// Creamos un diccionario donde la clave es el nombre del alumno.
var alumnos: [String: [Double]] = [:]

// Pedimos la cantidad de alumnos.
print("¿Cuántos alumnos desea registrar?")

// Convertimos la entrada del usuario a un número entero.
let cantidadAlumnos = Int(readline() ?? "") ?? 0

// Verificamos que la cantidad sea mayor que cero.
if cantidadAlumnos > 0 {

    // Repetimos el proceso para cada alumno.
    for i in 1...cantidadAlumnos {

        // Mostramos el número del alumno.
        print("\nAlumno \(i) - Nombre:")

        // Guardamos el nombre ingresado.
        let nombre = readline() ?? ""

        // Creamos un arreglo para almacenar las tres notas.
        var notas: [Double] = []

        // Indicamos que se deben ingresar tres notas.
        for j in 1...3 {

            // Pedimos la nota correspondiente.
            print("Nota \(j):")

            // Convertimos la nota ingresada a Double.
            let nota = Double(readline() ?? "") ?? 0

            // Guardamos la nota en el arreglo.
            notas.append(nota)

        }

        // Guardamos el nombre y sus tres notas en el diccionario.
        alumnos[nombre] = notas

    }

}
```

```
¿Cuántos alumnos desea registrar?
2

Alumno 1 - Nombre:
Haro Gabriel
Nota 1:
10
Nota 2:
12
Nota 3:
18

Alumno 2 - Nombre:
Rony Jhostin
Nota 1:
15
Nota 2:
17
Nota 3:
20
Rony Jhostin - Promedio: 17.33333333333332 - Bueno
Haro Gabriel - Promedio: 13.33333333333334 - Aprobado

===== ESTADÍSTICAS =====
Promedio general: 15.33333333333332
Nota más alta: 17.33333333333332
Nota más baja: 13.33333333333334
Porcentaje de aprobados: 100.0%
```

Referencia Rápida — Semana 3

Docente: Juan Leon — Tecsup

Ejercicio 7 :

```
1 import Foundation
2
3 // ===== EJERCICIO 7: INVENTARIO CON MENÚ =====
4
5 // Arreglo para guardar los nombres de los productos.
6 var nombres: [String] = []
7
8 // Arreglo para guardar los precios de los productos.
9 var precios: [Double] = []
10
11 // Arreglo para guardar el stock de los productos.
12 var stocks: [Int] = []
13
14 // Pedimos la cantidad de productos.
15 print("¿Cuántos productos desea registrar?")
16
17 // Leemos la cantidad de productos.
18 let cantidad = Int(readLine() ?? "") ?? 0
19
20 // Repetimos el proceso para cada producto.
21 for i in 1...cantidad {
22
23     // Pedimos el nombre del producto.
24     print("\nNombre del producto \(i):")
25
26     // Guardamos el nombre ingresado.
27     let nombre = readLine() ?? ""
28
29     // Pedimos el precio del producto.
30     print("Precio del producto \(i):")
31
32     // Convertimos el precio ingresado a Double.
33     let precio = Double(readLine() ?? "") ?? 0.0
34
35     // Pedimos el stock del producto.
36     print("Stock del producto \(i):")
37
38     // Convertimos el stock ingresado a Int.
39     let stock = Int(readLine() ?? "") ?? 0
40
41     // Agregamos el nombre al arreglo.
42     nombres.append(nombre)
43
44     // Agregamos el precio al arreglo.
45     precios.append(precio)
46
47     // Agregamos el stock al arreglo.
48     stocks.append(stock)
49 }
```

```
===== INVENTARIO =====
Producto: Arroz | Precio: S/ 4.50 | Stock: 20
Producto: Detergente BOLIVAR | Precio: S/ 7.80 | Stock: 5

===== MENÚ DE INVENTARIO =====
1. Ver inventario
2. Buscar producto
3. Stock bajo
4. Valor total
5. Salir

=====
Seleccione una opción:
3

===== STOCK BAJO =====
Producto: Detergente BOLIVAR | Stock: 5

===== MENÚ DE INVENTARIO =====
1. Ver inventario
2. Buscar producto
3. Stock bajo
4. Valor total
5. Salir

=====
Seleccione una opción:
4

===== VALOR TOTAL =====
Valor total: S/ 129.00
```

Concepto	Sintaxis	Ejemplo
readLine()	readLine() ?? ""	let e = readLine() ?? ""
Convertir	Double(texto) ?? 0	let d = Double(e) ?? 0
Array	var a: [Tipo] = []	var n: [Double] = []
append	.append(valor)	n.append(15.0)
remove	.remove(at: i)	n.remove(at: 0)
Dict	var d: [K:V] = [:]	var d: [String:Double] = [:]
Dict acceso	dict[clave]	d["Ana"]
if let	if let v = dict[k]	if let n = d["Ana"] {}
Set	var s: Set<T> = []	var s: Set<String> = []
intersection	.intersection(otro)	a.intersection(b)
union	.union(otro)	a.union(b)
subtracting	.subtracting(otro)	a.subtracting(b)

Rúbrica de Evaluación (sobre 20)

Docente: Juan Leon — Tecsup

Rama manual — Sin IA

Criterio	Excelente	Bueno (75%)	Regular (50%)	Deficiente
Ej 1: Arrays	TODO 1-3, FIX, PREDICT completos.	TODO sin FIX/PREDICT.	TODO incompletos.	Muy incompleto.
Ej 2: Diccionarios	TODO 4-7, ANALYZE completos.	TODO sin ANALYZE.	TODO incompletos.	No usa diccionarios.
Ej 3: Sets	TODO 8-9, PREDICT completos.	TODO sin PREDICT.	Solo crea Sets.	No usa Sets.
Ej 4: Combinación	TODO 10 completo. Inventario funcional.	Parcialmente completo.	Solo diccionarios.	No combina.
Ej 5: Carrito 2.0	TODO 11-18. Ticket completo.	Sin descuento o categoría.	Solo subtotales.	Muy incompleto.

Rama ai-assisted — Con IA

Criterio	Excelente	Bueno (75%)	Regular (50%)	Deficiente
Ej 6: Gestión	Sistema completo comentado.	Funcional sin comentarios.	Parcial.	Pegado sin entender.
Ej 7: Inventario	Menú while completo.	Sin menú o filtros.	Solo lista.	No funciona.
PROMPTS.md	CTRFE completa.	Sin estructura.	Incompleto.	No entrega.

Entregable

Docente: Juan Leon — Tecsup

- ❑ Rama manual → Ejercicios 1-5 interactivos con readLine()
- ❑ Rama manual → Header "// Desarrollado por: [NOMBRE]"
- ❑ Rama manual → Mínimo 6 commits progresivos
- ❑ Rama ai-assisted → Ejercicios 6-7 comentados línea por línea
- ❑ Rama ai-assisted → PROMPTS.md con estructura CTRFE
- ❑ Carrito 2.0 imprime ticket completo
- ❑ Todo ejecuta sin errores en Playground